BIENVENIDOS

Docker sin Rodeos

Empaqueta, Ejecuta y Despliega sin Complicaciones

```
$ docker run -d -p 8080:80 my-app
a1b2c3d4e5f67890 (Container Started)
CONTAINER ID IMAGE STATUS
a1b2c3d4e5f6 my-app Up 3 seconds
✓ Environment isolated & ready
```

Metodología de Enseñanza

Aprender haciendo con ejercicios reales, de la teoría a la práctica inmediata.

Espacio Interactivo

Preguntas abiertas en cualquier momento y resolución de dudas en tiempo real.

Objetivo de la Sesión

Domina las bases esenciales de Docker para empaquetar e independizar tus aplicaciones.

SEMANA 00

Del código al contenedor

Empaquetado, distribución y ejecución aislada de aplicaciones.



Imágenes

Plantillas de solo lectura con el entorno y código de la aplicación.

Contenedores

Instancias ejecutables y aisladas creadas a partir de una imagen.



Volúmenes

Mecanismos para la persistencia de datos fuera del contenedor.

Redes

Canales de comunicación aislados entre contenedores y host.

OBJETIVOS DE LA SESIÓN

¿Qué aprenderás hoy?

Metodología práctica centrada en aprender haciendo mediante laboratorios y retos en vivo.

FASE 01

1. Fundamentos Clave

Comprender los conceptos fundamentales de contenedores, imágenes, volúmenes y redes en Docker.

- ✔ Conceptos de contenedores
- ✔ Imágenes y Volúmenes
- ✔ Redes e Infraestructura

FASE 02

2. Laboratorios Prácticos

Ejercicios guiados paso a paso para desplegar y gestionar aplicaciones en entornos aislados reales.

- ✔ Guías paso a paso
- ✔ Despliegues en vivo
- ✔ Entornos aislados

FASE 03

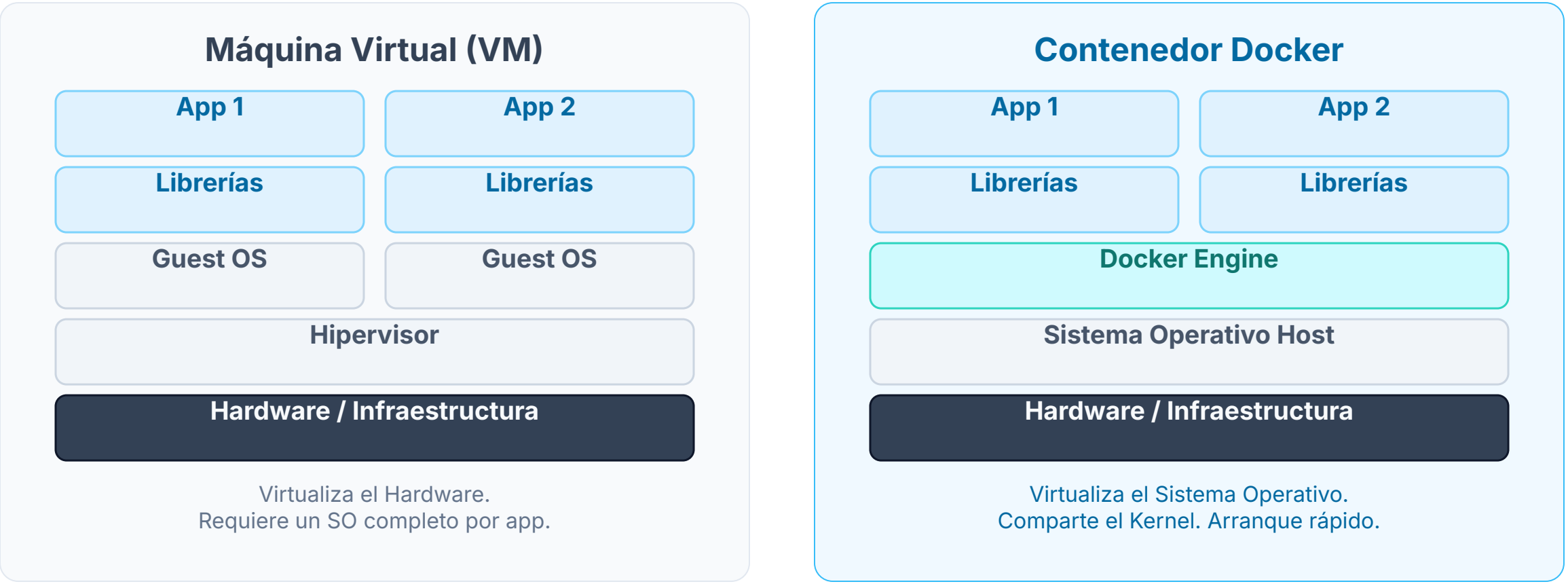
3. Retos Interactivos

Desafíos reales donde resolverás problemas complejos para consolidar el conocimiento "aprendiendo haciendo".

- ✔ Desafíos reales
- ✔ Resolución de problemas
- ✔ Aprendizaje práctico

Máquinas Virtuales vs Contenedores

Aislamiento ligero compartiendo el kernel del sistema operativo



El Motor Bajo el Capó

Las tecnologías de Linux que hacen posibles los contenedores



Namespaces

Aislamiento. Engañan al proceso para que crea que es el único en el sistema.

- **PID:** Árbol de procesos independiente.
- **NET:** Interfaces de red e IPs propias.
- **MNT:** Sistema de archivos aislado.



Control Groups (cgroups)

Límites. Evitan que un solo contenedor consuma todos los recursos del servidor.

- **CPU:** Limita el tiempo de procesamiento.
- **RAM:** Restringe la memoria máxima (OOM).
- **I/O:** Controla ancho de banda de discos.

Arquitectura Docker

Modelo Cliente-Servidor impulsado por API



El Ecosistema: Runtimes

Más allá de Docker en el mundo de Kubernetes

Docker Engine

Excelente para desarrollo local. Empaqueta todo (CLI, API, runtime, red). Sin embargo, es considerado "pesado" para ser orquestado directamente por Kubernetes a gran escala.

Containerd

El motor base de Docker extraído como un componente independiente y ligero. Es el runtime estándar actual en la inmensa mayoría de clústeres Kubernetes.

La Evolución hacia Kubernetes

Kubernetes → Dockershim → Docker → **Kubernetes (CRI) → Containerd**

Instrucciones Esenciales

El vocabulario principal para definir un contenedor

FROM

Define la imagen base inmutable. Obligatorio que sea la primera instrucción del archivo.

WORKDIR

Establece el directorio de trabajo interno para todas las instrucciones subsecuentes.

COPY

Copia archivos o directorios específicos desde el sistema host hacia el contenedor.

RUN

Ejecuta comandos de terminal en una nueva capa durante la fase de **construcción (build)**.

EXPOSE

Documenta el puerto de escucha previsto. No publica puertos reales por sí solo.

CMD

Define el proceso y comando principal por defecto al arrancar el contenedor.

Instrucciones: Diferencias Clave

Matices importantes y configuraciones de entorno

COPY vs ADD

- **COPY:** Copia archivos literalmente. Es la opción recomendada siempre.
- **ADD:** Extrae archivos .tar automáticamente y puede descargar desde URLs (menos predecible).

CMD vs ENTRYPOINT

- **ENTRYPOINT:** Define el ejecutable fijo. Difícil de sobrescribir al hacer docker run.
- **CMD:** Provee argumentos por defecto al Entrypoint. Se sobrescribe fácilmente por el usuario.

USER

Cambia el usuario (UID) para todas las instrucciones RUN, CMD y ENTRYPOINT siguientes. Vital para la seguridad (evitar procesos root).

ENV

Establece variables de entorno permanentes. **No debe usarse** para almacenar secretos (contraseñas/tokens), ya que quedan visibles en el historial.

Ejemplo: Node.js Dockerfile

Una receta declarativa para construir imágenes inmutables

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

EXPOSE 3000
CMD ["node", "server.js" ตอบ]
```

- **Estructura en capas:** Cada instrucción crea una capa inmutable y en caché.
- **Caché inteligente:** Si un archivo no cambia, Docker reutiliza la capa para acelerar la compilación.
- **Orden estratégico:** Copiar dependencias antes que el código fuente maximiza el uso de la caché.
- **.dockerignore:** Archivo vital para excluir ficheros innecesarios (como node_modules locales o logs).

Ejemplo: Python + Gunicorn

Adaptando las instrucciones a diferentes ecosistemas

- **Separar dependencias:** Copiar requirements.txt antes del código fuente para maximizar la caché.
- **Usuario dedicado:** Crear un usuario (appuser) para no ejecutar como root.
- **Servidor WSGI:** Usar un servidor listo para producción como Gunicorn en lugar del servidor de desarrollo local.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r
requirements.txt

COPY . .

RUN useradd -m appuser
USER appuser

CMD ["gunicorn", "--bind", "0.0.0.0:8000",
"app:app"]
```

Multi-Stage Builds

Separar compilación de ejecución para imágenes ultra ligeras

```
# Etapa 1: Compilación (SDK pesado)
FROM golang:1.21 AS builder
WORKDIR /src
COPY . .
RUN go build -o app_bin main.go

# Etapa 2: Ejecución (Base ultra ligera)
FROM alpine:latest
WORKDIR /app
# Copiar solo el binario compilado
COPY --from=builder /src/app_bin .

CMD ["./app_bin"]
```

El Problema

Los compiladores y herramientas (Go, JDK, Maven, GCC) son muy pesados, lo que aumenta drásticamente el tamaño y la superficie de vulnerabilidades de la imagen final.

La Solución Multi-Stage

Usar múltiples instrucciones FROM. Copias los artefactos (COPY --from=builder) hacia una imagen limpia. Resultado: una imagen de 10MB en lugar de 800MB.

Construcción y Ejecución

Ciclo de vida operativo básico

```
# 1. Construir la imagen desde el contexto actual (.)
docker build -t mi-api:v1 .

# 2. Ejecutar un contenedor basado en esa imagen
docker run -d --name servicio-api -p 8080:3000 mi-api:v1
```

`-d` (Detached)

Ejecuta el contenedor en segundo plano, liberando la terminal.

`--name`

Asigna un identificador fijo al contenedor en lugar de un hash aleatorio.

`-p 8080:3000`

Publica el puerto interno 3000 vinculándolo al puerto externo 8080 del host.

Operación de Contenedores

Comandos de gestión para el día a día

1. Estado y Listado

Verifica qué contenedores están corriendo en el host.

```
docker ps # Contenedores activos
docker ps -a # Incluye los detenidos
```

2. Ciclo de Vida

Apaga y elimina recursos (usar el ID o el nombre).

```
docker stop api-web
docker rm api-web # Elimina contenedor
docker rmi mi-api:v1 # Elimina imagen
```

3. Diagnóstico

Revisión de salidas estándar y metadatos internos.

```
docker logs -f api-web # Streaming
docker inspect api-web # JSON meta
```

4. Interacción (Shell)

Abre una terminal interactiva dentro del contenedor vivo.

```
# Entrar al contenedor
docker exec -it api-web /bin/sh
```

Variables de Entorno

Configuración dinámica (Factor 3 de las "12-Factor Apps")

- **Una imagen, múltiples entornos:** La misma imagen exacta debe poder ejecutarse en QA y Producción sin reconstruirse.
- **Separación de secretos:** Nunca escribas contraseñas en el Dockerfile ni en el código. Pásalas en tiempo de ejecución.
- **Flag -e:** Inyecta las variables directamente al arrancar el contenedor. (Base de ConfigMaps en K8s).

```
# Inyectar configuración al ejecutar
docker run -d \
   --name backend-api \
   -e ENVIRONMENT="production" \
   -e DB_HOST="10.0.0.5" \
   -e DB_PASS="s3cr3t!" \
   mi-api:v1
```

Manejo de Secretos

Protegiendo credenciales y tokens sensibles

Mala Práctica

Usar ENV o hardcodear secretos en el Dockerfile. Las variables de entorno son visibles mediante docker inspect y quedan expuestas en el historial de la imagen.

Buena Práctica

Montar secretos como archivos en memoria (tmpfs). En BuildKit usar --mount=type=secret. En ejecución, inyectarlos a través de gestores o mapeo de archivos de solo lectura.

```
# 1. En BuildKit: Secretos en compilación
RUN --mount=type=secret,id=aws_creds \
   cat /run/secrets/aws_creds > ~/.aws/credentials

# 2. En Ejecución (Montaje de solo lectura)
docker run -d --name app \
   -v /ruta/local/secret.txt:/run/secrets/db_pass:ro \
   mi-api:v1

# 3. En Docker Compose
secrets:
   db_password:
     file: ./secret_pass.txt
```

Distribución de Imágenes

El puente entre el entorno local y el clúster de Kubernetes



1. Local (Build)

Compilas la imagen y le asignas un tag (etiqueta) para el registry.

```
docker build -t app:v1 .
docker tag app:v1 repo/app:v1
docker images # Lista locales
```



2. Registry

Autenticación y carga (push) al almacén central público o privado.

```
docker login

docker push repo/app:v1
```



3. Destino (K8s)

El servidor descarga la imagen (pull) para poder ejecutarla.

```
docker pull repo/app:v1

docker run -d repo/app:v1
```

Persistencia de Datos

Evitar la pérdida de datos al destruir el contenedor efímero

Volúmenes (Volumes)

- Gestión por Docker Engine (Aislados).
- Ideales para BD en producción.

```
# 1. Crear y listar volúmenes
docker volume create pg_data
docker volume ls

# 2. Montar en contenedor
docker run -v pg_data:/var/lib/data ...
```

Montajes (Bind Mounts)

- Mapean rutas absolutas del Host.
- Ideales para desarrollo (Hot-reload).

```
# 1. No requiere creación previa

# 2. Montar ruta local al contenedor
docker run -v $(pwd)/src:/app/src ...

# Nota: Depende del OS anfitrión
```

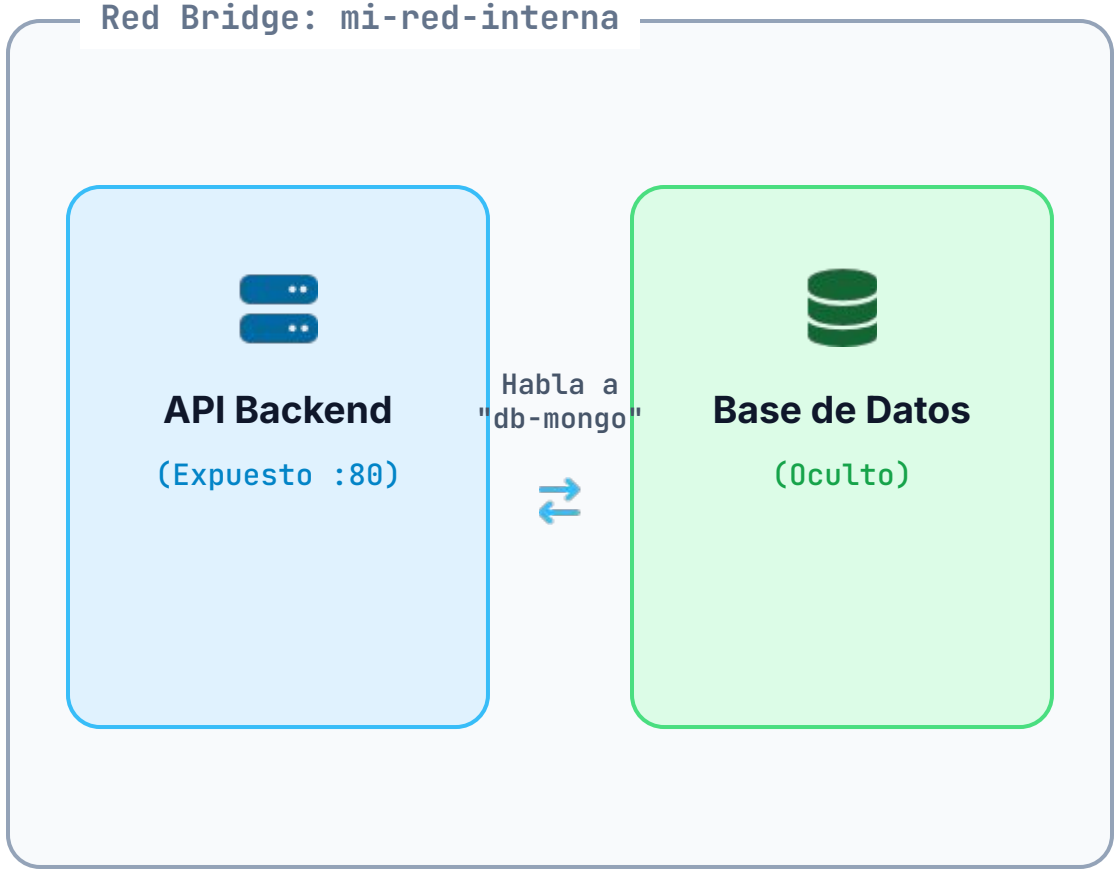
Redes y Descubrimiento

Aislamiento y comunicación interna segura

- **Bridge Personalizada:** Red aislada.
- **Resolución DNS:** Se comunican por nombre automáticamente (ej: db).

```
# 1. Crear y listar redes
docker network create mi-red
docker network ls

# 2. Conectar contenedores a la red
docker run -d --name db --network mi-red mysql
docker run -d --name api --network mi-red mi-api
```



Orquestación Local: Docker Compose

Infraestructura multi-contenedor como código

Declarativo

Reemplaza múltiples comandos run largos por un único archivo YAML legible.

Entornos Completos

Despliega servicios, redes y volúmenes simultáneamente con un solo comando.

Reproducibilidad

Permite versionar y compartir la arquitectura exacta con el equipo.

```
$ docker compose up -d   $ docker compose down
```

Anatomía de docker-compose.yml

Definición de servicios, dependencias y datos

```yaml
services:
api_backend:
build: ./backend
ports:
- "3000:3000"
environment:
- DB_HOST=database
depends_on:
- database

database:
image: postgres:15
volumes:
- pg_data:/var/lib/postgresql/data

volumes:
pg_data:
```

services: Define los contenedores individuales (api_backend, database).

depends_on: Controla el orden de inicio. La API espera a la base de datos.

environment: Inyecta variables de configuración. Se usa el nombre del servicio como Host.

volumes: Declara volúmenes gestionados a nivel superior para persistencia global.

Buenas Prácticas para Producción

Seguridad, eficiencia y minimalismo

Imágenes Base Ligeras

Usa variantes oficiales -slim o -alpine. Reducen drásticamente el tamaño y la superficie de ataque al tener menos paquetes de sistema operativo instalados.

Uso de .dockerignore

Crea un archivo estricto para excluir repositorios .git, dependencias locales (como node_modules), logs y secretos hardcodeados del contexto de construcción.

Usuario No Root

Aplica el principio de mínimo privilegio. Evita ejecutar procesos como administrador. Usa la instrucción USER para asignar una cuenta dedicada sin privilegios altos.

Multi-Stage Builds

Separa la compilación de la ejecución. Usa una imagen con SDK pesado para compilar, y copia solo el binario final a una imagen limpia de ejecución.

Análisis y Escaneo de Imágenes

Validar seguridad y optimización antes de pasar a producción

Escaneo de Vulnerabilidades

Detecta CVEs en las librerías del sistema operativo y dependencias de la aplicación (ej. npm audit).

```
docker scout cves app:v1

trivy image app:v1
```

Análisis de Capas (Layers)

Inspecciona qué archivos se agregan o modifican en cada capa para reducir el peso de la imagen.

```
docker history app:v1

dive app:v1
```

Linting de Dockerfile

Valida tu código fuente contra las mejores prácticas antes del comando de build.

```
hadolint Dockerfile

# Detecta uso de 'latest',
usuarios root, etc.
```

PRÓXIMO PASO

De Docker a Kubernetes



Límite Local

Docker empaqueta y ejecuta en un solo nodo.

¿Qué pasa si el servidor físico cae?



Orquestación Distribuida

Kubernetes gestiona flotas enteras de contenedores en múltiples servidores (clúster).



Estado Declarativo

Defines cuántas réplicas necesitas, y K8s se asegura de mantenerlas vivas automáticamente.

→ **Preparados para explorar.**